Iteration B2: Unit Testing of Models

One of the joys of the Rails framework is that it has support for testing baked right in from the start of every project. As you've seen, from the moment you create a new application using the `rails` command, Rails starts generating a test infrastructure for you. Let's take a peek inside the `models` subdirectory to see what's already there:

```
depot> ls test/models
product_test.rb
```

`product_test.rb` is the file that Rails created to hold the unit tests for the model we created earlier with the `generate` script. This is a good start, but Rails can help us only so much. Let's see what kind of test goodies Rails generated inside `test/models/product_test.rb` when we generated that model:

rails7/depot_a/test/models/product_test.rb

```
require "test_helper"

class ProductTest < ActiveSupport::TestCase
  # test "the truth" do
  #   assert true
  # end
end
```

The generated `ProductTest` is a subclass of `ActiveSupport::TestCase`.[40] The fact that `ActiveSupport::TestCase` is a subclass of the `MiniTest::Test` class tells us that Rails generates tests based on the MiniTest[41] framework that comes preinstalled with Ruby. This is good news, because it means if we've already been testing our Ruby programs with MiniTest tests (and why wouldn't we be?), we can build on that knowledge to test Rails applications. If you're new to MiniTest, don't worry. We'll take it slow.

Inside this test case, Rails generated a single commented-out test called `"the truth"`. The `test...do` syntax may seem surprising at first, but here `ActiveSupport::TestCase` is combining a class method, optional parentheses, and a